MIST 4610 Group Project 1

Team Name:

[Your Section Number] Group [Your Group Number]

Team Members:

1. [Member Name] @username
2. [Member Name] @username
3. [Member Name] @username
4. [Member Name] @username

Problem Description:

The task at hand is to model and build a relational database for a property management company operating multiple residential properties in Athens, GA. The central entity in the model is the Properties entity, which represents each physical apartment complex the company owns and manages. Each property contains multiple units that are leased to tenants. The company tracks lease agreements, rent payments, maintenance requests, vendor assignments, parking spot allocations, pet registrations, and amenity bookings. We are interested in accurately modeling these relationships, generating sample data, and populating the entities and their attributes with this data. Furthermore, we are interested in performing functioning queries on this data so that they may provide us with valuable business insight about the properties and their operations.

Data Model:

Explanation of data model:

Our model is based on the structure of a property management company that oversees residential apartment complexes.

The Properties entity represents each apartment complex the company manages. Each property has a name, address, total number of units, and a building age. A property contains many units, represented by the one-to-many relationship between Properties and Units. Each unit has a bedroom count, bathroom count, and occupancy status.

Tenants reside in units through lease agreements. Because a tenant can have multiple leases over time, and a unit can be leased to different tenants, the Leases entity serves as the link between Units and Tenants. Each lease records the start date, end date, and monthly rent amount.

Tenants make monthly rent payments, which are tracked in the Payments entity. Each payment records the amount, date, payment method, and whether the payment was completed, pending, or failed. Payments are tied back to a specific tenant, unit, and lease.

When something breaks in a unit, a tenant submits a maintenance request. The MaintenanceRequests entity records the issue description, date reported, and current status (Open, In Progress, Resolved, or Urgent). Each request is assigned to a vendor from the Vendors entity, which stores the vendor's name and service category (Plumbing, Electrical, HVAC, etc.).

Each unit is assigned a parking spot tracked in the ParkingSpots entity, which records the spot number and type (Covered, Uncovered, or Garage).

Tenants may register pets, which are stored in the Pets entity with the pet's name, type (Dog or Cat), and breed. Each pet is linked to a specific tenant.

Finally, tenants can book shared amenities (pools, gyms, clubhouses, etc.) through the AmenityBookings entity, which records the booking date and time slot.

(Insert your data model PNG image here)

Data Dictionary:

(Insert your data dictionary table screenshots here — see the example project for formatting)

Queries:

Feature Matrix

Feature	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
Multiple Table JOIN	✓		✓	✓		✓	✓	✓	✓	✓
Traditional Subquery			✓							
Correlated Subquery		✓		✓						
GROUP BY	✓		✓		✓			✓		
GROUP BY with HAVING	✓		✓							
Multi-condition WHERE				✓		✓			✓	
Built-in Functions / Calculated Field	✓		✓	✓	✓			✓		
REGEXP						✓				
NOT EXISTS		✓								

COMPLEX QUERIES

Query 1 lists the name of each property, the number of completed payments received, and the total revenue collected from those payments. Only properties that have collected more than $2,000 in total completed payments are shown. Results are ordered by total revenue in descending order.

```sql
SELECT p.propertyName,
       COUNT(py.paymentID) AS numPayments,
       SUM(py.amountPaid) AS totalRevenue
FROM Properties p
JOIN Units u ON p.propertyID = u.propertyID
JOIN Payments py ON u.unitID = py.unitID
WHERE py.Status = 'Completed'
GROUP BY p.propertyName
HAVING SUM(py.amountPaid) > 2000
ORDER BY totalRevenue DESC;
```

```
propertyName    numPayments  totalRevenue
--------------  -----------  ------------
Athens Lofts    4            5400
Parkside Lofts  2            3000
The Standard    2            2250
```

Query 1 allows management to quickly identify the top revenue-generating properties. A property manager overseeing 21 properties needs to know which locations are bringing in the most money so they can allocate maintenance budgets, staffing, and marketing resources accordingly. Properties not appearing on this list may need attention — either they have low occupancy or tenants are behind on payments.

Query 2 lists all tenants who have never submitted a maintenance request.

```sql
```

```sql
SELECT t.tenantID, t.firstName, t.lastName
FROM Tenants t
WHERE NOT EXISTS (
    SELECT 1 FROM MaintenanceRequests mr
    WHERE mr.tenantID = t.tenantID
)
ORDER BY t.lastName;
```

```
tenantID  firstName  lastName
--------  ---------  --------
19        Samuel     Allen
9         Ian        Hall
18        Rachel     Hall
21        Ulysses    King
8         Hannah     Walker
10        Julia      Young
20        Taylor     Young
```

Query 2 allows management to identify tenants who have never reported any issues. This could mean two things: either those tenants are satisfied, or they are not reporting problems that could be getting worse over time. A proactive manager could reach out to these tenants for a satisfaction check or schedule a routine unit inspection to catch hidden issues like water damage or pest problems before they become expensive.

Query 3 lists the vendors who have handled more maintenance requests than the average number of requests per vendor, along with their service category and total request count.

```sql
```

```sql
SELECT v.vendorName, v.serviceCategory,
       COUNT(mr.requestID) AS totalRequests
FROM Vendors v
JOIN MaintenanceRequests mr ON v.vendorID = mr.vendorID
GROUP BY v.vendorID, v.vendorName, v.serviceCategory
HAVING COUNT(mr.requestID) > (
    SELECT AVG(reqCount)
    FROM (
        SELECT COUNT(requestID) AS reqCount
        FROM MaintenanceRequests
        GROUP BY vendorID
    ) AS vendorCounts
)
ORDER BY totalRequests DESC;
```

```
vendorName            serviceCategory      totalRequests
--------------------  -------------------  -------------
Quick Fix Appliances  Appliance Repair     3
Apex Plumbing         Plumbing             2
Solid Foundations     General Contracting  2
Secure Locksmiths     Security             2
Pro Paint Crew        Painting             2
```

Query 3 allows management to identify which vendors are being relied on the most. Vendors with above-average workloads (like Quick Fix Appliances with 3 requests) may need contract renegotiation for volume discounts, or management may want to evaluate whether they should bring in a second vendor in that category to distribute the work. This also helps with budget forecasting — high-demand service categories like Appliance Repair clearly represent a recurring cost center.

Query 4 lists the properties whose unit occupancy rate falls below the overall company average of 71.4%. The query calculates the number of occupied units and the occupancy percentage for each qualifying property.

```sql
```

```sql
SELECT p.propertyID, p.propertyName,
       (SELECT COUNT(*) FROM Units u WHERE u.propertyID = p.propertyID) AS totalUnit
       (SELECT COUNT(*) FROM Units u WHERE u.propertyID = p.propertyID
        AND u.isOccupied = 'Yes') AS occupiedUnits,
       ROUND(
         (SELECT COUNT(*) FROM Units u WHERE u.propertyID = p.propertyID
          AND u.isOccupied = 'Yes') * 100.0 /
         (SELECT COUNT(*) FROM Units u WHERE u.propertyID = p.propertyID), 1
       ) AS occupancyPct
FROM Properties p
WHERE (SELECT COUNT(*) FROM Units u WHERE u.propertyID = p.propertyID) > 0
  AND (SELECT COUNT(*) FROM Units u WHERE u.propertyID = p.propertyID
       AND u.isOccupied = 'Yes') * 100.0 /
      (SELECT COUNT(*) FROM Units u WHERE u.propertyID = p.propertyID)
      < (SELECT SUM(CASE WHEN isOccupied = 'Yes' THEN 1 ELSE 0 END) * 100.0
         / COUNT(*) FROM Units)
ORDER BY occupancyPct ASC;
```

propertyID	propertyName	totalUnitsInDB	occupiedUnits	occupancyPct
6	River Mill	1	0	0.0
12	University Oaks	1	0	0.0
15	Lakeside Living	1	0	0.0
19	Heritage House	1	0	0.0
2	Oconee Heights	2	1	50.0
3	Classic City Flats	2	1	50.0

Query 4 is essential for any property manager because vacant units directly equal lost revenue. By comparing each property's occupancy to the company-wide average, management can immediately see which properties are underperforming. River Mill, University Oaks, Lakeside Living, and Heritage House all have 0% occupancy — these need urgent attention such as rent reductions, marketing, or building improvements to attract tenants.

Query 5 lists each payment method used by tenants, the total number of transactions made with that method, the number of completed and failed transactions, and the failure rate as a percentage. Results are ordered by failure rate in descending order.

sql

```sql
SELECT paymentMethod,
       COUNT(*) AS totalTransactions,
       SUM(CASE WHEN Status = 'Completed' THEN 1 ELSE 0 END) AS completed,
       SUM(CASE WHEN Status = 'Failed' THEN 1 ELSE 0 END) AS failed,
       ROUND(SUM(CASE WHEN Status = 'Failed' THEN 1 ELSE 0 END) * 100.0
             / COUNT(*), 1) AS failureRate
FROM Payments
GROUP BY paymentMethod
ORDER BY failureRate DESC;
```

```
paymentMethod  totalTransactions  completed  failed  failureRate
-------------  -----------------  ---------  ------  -----------
Check          4                  0          2       50.0
Debit Card     3                  3          0       0.0
Credit Card    8                  8          0       0.0
Bank Transfer  6                  6          0       0.0
```

Query 5 reveals that checks have a 50% failure rate — by far the most unreliable payment method — while Credit Card, Debit Card, and Bank Transfer have zero failures. A property manager can use this to justify changing the company's payment policy, such as adding a processing fee for checks or requiring tenants to switch to electronic payment methods. Failed payments create accounting overhead and delay cash flow, so reducing check usage would directly improve operational efficiency.

Query 6 lists tenants whose last names match common surname patterns (Clark, Hall, Walker, or Lewis), along with their monthly rent and property name, but only for tenants whose leases are still active (ending on or after April 1, 2024).

sql

```sql
SELECT t.tenantID, t.firstName, t.lastName, l.monthlyRent, p.propertyName
FROM Tenants t
JOIN Leases l ON t.tenantID = l.tenantID
JOIN Units u ON l.unitID = u.unitID
JOIN Properties p ON u.propertyID = p.propertyID
WHERE t.lastName REGEXP '(Clark|Hall|Walker|Lewis)'
  AND l.endDate >= '2024-04-01'
ORDER BY t.lastName, t.firstName;
```

```
tenantID  firstName  lastName  monthlyRent  propertyName
--------  ---------  --------  -----------  --------------------
6         Fiona      Clark     1600         Bulldog Village
13        Matthew    Clark     1250         Pineview Apartments
7         George     Lewis     950          Downtown Suites
15        Oliver     Lewis     1550         Parkside Lofts
17        Quinn      Walker    1350         Campus View
```

Query 6 demonstrates REGEXP for flexible pattern matching, which is useful in database management for finding tenants based on name patterns. In practice, a property manager might use this type of query to pull up all tenants from a certain family (e.g., multiple Clarks or Lewises who may be related), to cross-reference names for duplicate account detection, or to quickly filter a large tenant list during targeted outreach campaigns such as lease renewal reminders.

SIMPLE QUERIES

Query 7 lists every tenant with an active lease (ending in 2024 or later), the number of bedrooms and bathrooms in their unit, and the property they live in.

sql

```sql
SELECT t.firstName, t.lastName, u.Bedrooms, u.Bathrooms, p.propertyName
FROM Tenants t
JOIN Leases l ON t.tenantID = l.tenantID
JOIN Units u ON l.unitID = u.unitID
JOIN Properties p ON u.propertyID = p.propertyID
WHERE l.endDate >= '2024-01-01'
ORDER BY p.propertyName;
```

```
firstName  lastName   Bedrooms  Bathrooms  propertyName
---------  ---------  --------  ---------  --------------------
Alice      Smith      1         1          Athens Lofts
Bob        Johnson    2         2          Athens Lofts
Penelope   Lee        3         2          Athens Skyline
Fiona      Clark      2         2          Bulldog Village
Quinn      Walker     2         2          Campus View
Diana      Evans      2         1.5        Classic City Flats
George     Lewis      1         1          Downtown Suites
Laura      Robinson   3         3          Georgia Heights
Charlie    Davis      1         1          Oconee Heights
Oliver     Lewis      2         2          Parkside Lofts
Matthew    Clark      2         1.5        Pineview Apartments
Ethan      Harris     1         1          The Archway
Kevin      Martinez   2         2          The Foundry
Natalie    Rodriguez  1         1          The Standard
```

Query 7 gives management a complete roster of all current tenants, where they live, and what type of unit they occupy. This is a fundamental operational query — it answers the basic question of "who lives where right now?" which is needed for communications, emergency contacts, lease renewal planning, and ensuring every occupied unit is accounted for in the system.

Query 8 lists the number of leases and the average monthly rent for each bedroom count (1-bedroom, 2-bedroom, 3-bedroom).

```sql
SELECT u.Bedrooms,
       COUNT(l.leaseID) AS numLeases,
       ROUND(AVG(l.monthlyRent), 2) AS avgMonthlyRent
FROM Units u
JOIN Leases l ON u.unitID = l.unitID
GROUP BY u.Bedrooms
ORDER BY u.Bedrooms;
```

```
Bedrooms  numLeases  avgMonthlyRent
--------  ---------  --------------
1         8          1112.5
2         10         1420.0
3         3          1783.33
```

Query 8 helps management understand their pricing structure. It shows that 2-bedroom units are the most commonly leased (10 leases), while 3-bedroom units command the highest average rent at $1,783. A manager could use this to evaluate whether their pricing is competitive with the local Athens rental market, or to decide which unit types to prioritize in future property acquisitions or renovations.

Query 9 lists all maintenance requests that are not yet resolved (Open, In Progress, or Urgent), along with the assigned vendor and their service category. Results are sorted by the date the issue was reported.

```sql
SELECT mr.requestID, mr.issueDescription, mr.Status, mr.dateReported,
       v.vendorName, v.serviceCategory
FROM MaintenanceRequests mr
JOIN Vendors v ON mr.vendorID = v.vendorID
WHERE mr.Status IN ('Open', 'In Progress', 'Urgent')
ORDER BY mr.dateReported;
```

```
requestID  issueDescription              Status       dateReported  vendorName
serviceCategory
---------  ----------------------------  -----------  ------------  --------------
------  --------------------
2          AC not cooling                In Progress  2024-02-20    Cool Breeze
HVAC      HVAC
24         Drafty windows                Open         2024-03-05    Pro Paint Crew
Painting
3          Broken window blind           Open         2024-03-10    Solid
Foundations     General Contracting
5          Outlets in bedroom sparking   Urgent       2024-03-15    Current
Electric      Electrical
7          Ants in the kitchen           Open         2024-03-18    Pest Control
Pros     Pest Control
11         Roof leaking                  In Progress  2024-03-20    Athens Roofing
Pros   Roofing
12         Peeling paint in living room  Open         2024-03-22    Pro Paint Crew
Painting
17         Broken fence panel            In Progress  2024-03-25    Ironclad
Fencing      Fencing
14         Internet goes out randomly    Open         2024-04-02    Tech Connect
IT Services
25         Sprinkler head broken         In Progress  2024-04-08    Green Leaf
Tree Care  Landscaping
```

Query 9 is the maintenance team's daily dashboard. It shows every outstanding issue, how long it has been open, and which vendor is responsible. The "Outlets in bedroom sparking" issue marked as Urgent is a safety hazard that demands immediate follow-up. The AC issue has been open since February — the longest unresolved request — which may indicate vendor delays. A manager can use this query to hold vendors accountable for response times and prioritize work orders.

Query 10 lists all tenants who own pets, along with the pet's name, type, breed, and the property the tenant currently lives in. Only tenants with active leases are included.

```sql
```

```sql
SELECT t.firstName, t.lastName, pt.petName, pt.petType, pt.Breed, p.propertyName
FROM Tenants t
JOIN Pets pt ON t.tenantID = pt.Tenants_tenantID
JOIN Leases l ON t.tenantID = l.tenantID
JOIN Units u ON l.unitID = u.unitID
JOIN Properties p ON u.propertyID = p.propertyID
WHERE l.endDate >= '2024-01-01'
ORDER BY t.lastName;
```

firstName	lastName	petName	petType	Breed	propertyName
Fiona	Clark	Max	Dog	German Shepherd	Bulldog Village
Matthew	Clark	Buster	Dog	Bulldog	Pineview Apartments
Charlie	Davis	Chloe	Cat	Ragdoll	Oconee Heights
Diana	Evans	Charlie	Dog	Golden Retriever	Classic City Flats
Ethan	Harris	Milo	Cat	Maine Coon	The Archway
Bob	Johnson	Luna	Cat	Siamese	Athens Lofts
Penelope	Lee	Nala	Cat	Scottish Fold	Athens Skyline
Oliver	Lewis	Sadie	Dog	Dalmatian	Parkside Lofts
Kevin	Martinez	Rocky	Dog	Boxer	The Foundry
Laura	Robinson	Smokey	Cat	Russian Blue	Georgia Heights
Natalie	Rodriguez	Simba	Cat	Sphynx	The Standard
Alice	Smith	Bella	Dog	Labrador Retriever	Athens Lofts
Alice	Smith	Daisy	Dog	Poodle	Athens Lofts
Quinn	Walker	Toby	Dog	Cocker Spaniel	Campus View

Query 10 allows management to see exactly which tenants have pets and at which properties. This is important for ensuring pet deposits have been collected, verifying that pets comply with breed or size restrictions in the lease, and planning pet-friendly amenities or services. It also helps with property inspections — units with large dog breeds like German Shepherds or Dalmatians may experience more wear and tear than units with cats.

Database information:

Name of the database: ns_[your section]_[your group number]

Additional information: Each query listed above is marked in the database using stored procedures which can be called using the following format: CALL TP_Q1();